

# Supply dispatch audit

## Practical Lab 1 · C++ foundations · Independent assessment

A game-support team needs a console tool to audit supplies before dispatching them. A depot can request more units than it has in stock, input can be mistyped, and a batch file can be incomplete. Your program must report what can actually be dispatched without confusing an invalid file with a valid small stockholding.

Build the program from this specification. A separate, deliberately faulty project tests whether you can diagnose build failures. There is no worked implementation to follow.

**Build and diagnose** — separate files, declarations and definitions.

**Read and validate** — console responses and a bounded batch file.

**Update and explain** — scalar values, pointer targets and safe access.

The two-hour target is easier to manage if you use the following rough plan. These are planning guides, not time limits.

| Stage                       | Suggested time | Ready to move on when...                                                                         |
|-----------------------------|----------------|--------------------------------------------------------------------------------------------------|
| Set-up                      | 5 min          | You have opened the correct solution and know where the active <code>batches.txt</code> file is. |
| A · Build and diagnose      | 20 min         | Both projects build; <code>BuildInvestigation</code> runs and prints <code>Packages: 3</code> .  |
| B · Console input           | 25 min         | A full depot name is retained; invalid requests recover; a valid request is accepted.            |
| C · Batch audit             | 30 min         | The whole file is validated before success; baseline gives 4 batches, 40 units, 2 low batches.   |
| D · Dispatch and pointers   | 25 min         | The four published dispatch cases match and the pointer is safely reset.                         |
| E · Evidence and submission | 15 min         | Eight tests are recorded, baseline data is restored and <code>evidence.md</code> is complete.    |

## Your working agreement

Allow approximately **two hours** for the whole assessment. Complete Tasks A–E; all are included in the **100 marks**. Basic, intermediate and expert describe the quality of your evidence, not different worksheets or extra topics. A partial implementation can earn credit for requirements it genuinely meets.

Use **Windows, Visual Studio, C++17, Debug, x64 and /W4**, with GitHub access already available. The supplied projects target the v143 toolset; retarget to an installed compatible toolset when necessary. Opening a project and resolving an installation problem are not additional programming challenges.

Work independently under the collaboration, documentation and AI conditions published with the Blackboard assessment. Write your own implementation rather than transplanting a completed tutorial. Supplied data and project configuration may be reused. Appropriate assistive input remains permitted under your approved arrangements.

## What you are demonstrating

You will **implement** a stream-based program, **diagnose** compilation and linking failures, **trace** changes through a non-owning pointer, and **justify** tests against a stated contract. Here, a **contract** means the input rules and required behaviour against which your program is assessed.

No arrays, reference parameters, enumerations, custom classes, dynamic allocation, file output or game engine are required. Use the scalar variables, conditions, loops and functions already available to you.

# A · Build a maintainable program and diagnose faults

15 marks · Required artefacts: two repaired/implemented projects and a short diagnostic record

The `Lab_01` folder contains two separate solutions. Keep them separate; do not add both `Main.cpp` files to one executable project. The source files in `DispatchAudit` are intentionally unimplemented. The `BuildInvestigation` files contain a small diagnostic specimen, not a solution to the audit.

## A1 · Your audit project

Use the following organisation. The heading wording is your choice, but the function must be called by the executable.

| File                                    | Required responsibility                                                                         |
|-----------------------------------------|-------------------------------------------------------------------------------------------------|
| <code>DispatchAudit/Main.cpp</code>     | One <code>main</code> function; console input, file processing and the dispatch demonstration   |
| <code>DispatchAudit/Dispatch.h</code>   | Guarded declaration of <code>void printHeading();</code> inside namespace <code>Dispatch</code> |
| <code>DispatchAudit/Dispatch.cpp</code> | The matching definition of <code>Dispatch::printHeading</code>                                  |
| <code>DispatchAudit/batches.txt</code>  | The active input data, beside the project file                                                  |
| <code>evidence.md</code>                | Your diagnostic record, test results and explanations                                           |

Use standard-library names with `std::`. Do not place a `using namespace std;` directive in the header. Initialise variables before use and use consistent four-space indentation. Comments should explain decisions or rules, not narrate every assignment.

### A1 · Work in this order

**Step 1 - Open only the audit solution.** Open `DispatchAudit` and confirm that it is the project you are building. Keep `BuildInvestigation` separate; it has its own `Main.cpp`.

**Step 2 - Confirm the build configuration.** Use Debug and x64, C++17 and /W4. If Visual Studio cannot use v143, retarget to an installed compatible toolset before treating that as a programming error.

**Step 3 - Make the interface compile.** In `Dispatch.h`, keep a header guard and declare `void printHeading();` inside namespace `Dispatch`. In `Dispatch.cpp`, include the header and provide the matching definition.

**Step 4 - Call the function from `Main.cpp`.** Include `Dispatch.h` and call `Dispatch::printHeading()` once. Build and run now. Do not wait for console input or file processing to be finished before checking this milestone.

**Step 5 - Confirm the data-file location.** The program will later open `batches.txt` relative to its working directory. Check that the physical file is beside the project file as supplied; seeing a file in Solution Explorer does not prove that the executable can open it.

**Checkpoint A1:** `DispatchAudit` builds and running it prints your heading. The rest of the audit may still be unfinished. If this checkpoint does not work, fix project structure before continuing.

## A2 · Investigate the supplied build

The intended `BuildInvestigation` executable prints its diagnostic heading followed by `Packages: 3`. It contains **two independent faults**. Diagnose and repair both without changing that intended behaviour or removing the call to `Banner::show`.

For each fault, record the first useful diagnostic, the stage that detected it, the file and cause, and the change that repaired it. Exact diagnostic codes may differ between compiler versions. Include a final successful build and output record; do not run an old executable after a failed build.

### A2 · A repeatable diagnosis workflow

**Step 1 - Build before editing.** Trigger a fresh build of `BuildInvestigation` and read the first useful diagnostic. Later messages may be consequences of the first fault.

**Step 2 - Classify the stage.** Decide whether the diagnostic comes from compilation of one source file or from linking the already-compiled translation units. Record that stage in `evidence.md`.

**Step 3 - Repair one cause at a time.** Change only what is needed for the first fault, build again, then investigate the next independent fault. This keeps your diagnostic record understandable.

**Step 4 - Run the newest executable.** After the final successful build, run the project and compare the output with the required behaviour.

**Expected final `BuildInvestigation` result:** The supplied diagnostic heading is followed by the line below. A successful build alone is not the final check.

```
Packages: 3
```

**Diagnostic record template:** Use one compact record per fault.

```
Fault 1 | first useful diagnostic | compiler/linker | file + cause | repair | result
Fault 2 | first useful diagnostic | compiler/linker | file + cause | repair | result
```

## C · Recommended file-reading sequence

**Step 1 - Use the active file.** The program always opens `DispatchAudit/batches.txt`. When testing supplied data cases, put the chosen case into that active file rather than changing the filename in your code for every test.

**Step 2 - Open and check.** Create a `std::ifstream` for `batches.txt`. If it cannot be opened, report an open failure and return a non-zero status before printing a completed report.

**Step 3 - Initialise the audit totals.** Start `batchCount`, `totalStock` and `lowBatchCount` at zero. These totals only become a successful audit if the whole file is valid.

**Step 4 - Separate clean end-of-file from invalid data.** Before each integer, consume whitespace with `std::ws`. If that reaches a normal end-of-file, the sequence is complete. Otherwise attempt one integer extraction; if that extraction fails, the file contains invalid data.

**Step 5 - Validate before adding.** For every extracted integer, reject it if it is outside 0–40. Increment the entry count and reject a thirteenth entry. Only then add the value to `totalStock` and update the low-batch count for values below 10.

**Step 6 - Finish the file audit first.** After a clean end, reject the file if `batchCount` is still zero. Do not calculate or apply a dispatch until all entries have been read and validated.

**Why the pre-check matters:** A simple loop such as `while (file >> value)` followed only by “if eof, success” can mistake a malformed final token for a clean ending. Consuming whitespace and checking for a clean end before the next integer attempt keeps “12 8 +” invalid, as required.

| Example active <code>batches.txt</code> contents | Expected interpretation                                         |
|--------------------------------------------------|-----------------------------------------------------------------|
| 12 0 8 20                                        | Valid: 4 batches, total 40, low batches 2.                      |
| 4 6                                              | Valid: 2 batches, total 10, low batches 2.                      |
| 0 0                                              | Valid: 2 batches, total 0, low batches 2.                       |
| (empty or whitespace only)                       | No-entries error; non-zero exit; no completed report.           |
| 12abc 8                                          | Invalid-data error; do not silently discard <code>abc</code> .  |
| 12 8 +                                           | Invalid-data error, even though the failed token is at the end. |
| 41                                               | Invalid-data error because the value is outside 0–40.           |
| 13 valid-looking integers                        | Invalid-data error because a thirteenth entry is not permitted. |

## D · Implement the dispatch as observable state changes

**Step 1 - Calculate one dispatched amount.** Use the smaller of requested units and original stock. A request of zero gives zero dispatched units; a shortage uses all available stock and is still a valid dispatch result.

**Step 2 - Keep the original total.** Leave `totalStock` unchanged. Set `availableStock = totalStock` and `dispatchCount = 0` before pointer-based updates begin.

**Step 3 - Select stock and update through the pointer.** Make `selectedQuantity` point to `availableStock`, then subtract the dispatched amount through `*selectedQuantity`. Do not subtract directly from `totalStock`.

**Step 4 - Retarget and count a non-empty dispatch.** Change the same pointer so that it points to `dispatchCount`. Increment `*selectedQuantity` exactly once only when dispatched units are greater than zero.

**Step 5 - End the selection safely.** Set `selectedQuantity` to `nullptr`. Check for `nullptr` before any dereference and print a clear no-selection message on that guarded path.

**One acceptable baseline report:** Labels and spacing may differ, but the values must have the same meaning.

```
Depot: North Depot
Batches: 4
Low batches: 2
Original stock: 40
Requested units: 25
Dispatched units: 25
Remaining stock: 15
Dispatch count: 1
No quantity selected.
```

## E · How to run and record each test

**Step 1 - Predict first.** Before running the program, write the expected result from the published contract. This makes the test evidence about reasoning, not just screenshots.

**Step 2 - Prepare the active data file.** For a supplied data case, copy its contents into the active `DispatchAudit/batches.txt`. For P5, temporarily make `batches.txt` unavailable. Do not change your C++ filename just to make a test easier.

**Step 3 - Run from a fresh start.** Build if the source changed, then start a `new` run and enter the stated console responses. Record whether the program finishes normally or reports an error.

**Step 4 - Compare actual with predicted.** Record the important values or error category and mark the test Pass only if the observed behaviour matches the acceptance condition. If it fails, record the defect honestly and fix it if time permits.

**Step 5 - Restore the baseline.** After testing, restore `batches.txt` to 12, 0, 8, 20 on separate lines so the submitted project starts from the published baseline.

| ID | Prediction before run | Actual result | Pass? | Evidence / defect note |
|----|-----------------------|---------------|-------|------------------------|
| P1 | ...                   | ...           | ...   | ...                    |
| P2 | ...                   | ...           | ...   | ...                    |
| P3 | ...                   | ...           | ...   | ...                    |
| P4 | ...                   | ...           | ...   | ...                    |
| P5 | ...                   | ...           | ...   | ...                    |
| P6 | ...                   | ...           | ...   | ...                    |
| P7 | ...                   | ...           | ...   | ...                    |
| P8 | ...                   | ...           | ...   | ...                    |

**Good P8 choices:** Choose one published edge case that is not already P1–P7: whitespace-only input, a value of 41, exactly 12 entries, 13 entries, no final newline, zero-stock data, or ended console input. State which contract rule it probes and why the baseline case would not detect that mistake.

## D · Baseline pointer-trace self-check

Use your own observed addresses. The values below are the expected logical state for baseline stock 40 and request 25.

| Trace point                         | <code>selectedQuantity</code> points to | <code>availableStock</code> | <code>dispatchCount</code> | Target value  |
|-------------------------------------|-----------------------------------------|-----------------------------|----------------------------|---------------|
| Before stock update                 | <code>availableStock</code>             | 40                          | 0                          | 40            |
| After stock update                  | <code>availableStock</code>             | 15                          | 0                          | 15            |
| After retarget + possible increment | <code>dispatchCount</code>              | 15                          | 1                          | 1             |
| After <code>nullptr</code> reset    | nothing ( <code>nullptr</code> )        | 15                          | 1                          | not evaluated |

**Expression check:** `selectedQuantity` is the pointer's stored address; `*selectedQuantity` is the value in the selected target object; `&selectedQuantity` is the address of the pointer variable itself. These are three different values/expressions. After `nullptr`, do not evaluate `*selectedQuantity`.

**Checkpoint C:** With the supplied baseline file, stop and verify `batchCount = 4`, `totalStock = 40` and `lowBatchCount = 2` before writing the dispatch logic.

### A3 · Explain the build you produced

In no more than **80 words**, identify the language standard, compiler and development environment you used. Explain how the header declaration and source-file definition let `Main.cpp` call the function, and distinguish an ordinary incremental Build from Rebuild. Use your own file names in the explanation.

#### A3 · 80-word explanation checklist

**Include four things:** (1) C++17, compiler and Visual Studio environment; (2) which header declares the function; (3) which `.cpp` file defines it and how `Main.cpp` sees the declaration; and (4) the practical difference between an incremental Build and Rebuild. Four short sentences are enough.

**Sentence prompts:** Write the explanation in your own words rather than copying these prompts.

I built the project using ...

`Main.cpp` can call ... because ...

The matching definition is in ... and is resolved when ...

Build ..., whereas Rebuild ...

**Evidence boundary:** a successful build shows that compilation and linking completed. Your later tests must establish whether the program meets the data and dispatch requirements.

## B · Accept a depot name and a dispatch request

20 marks · Required behaviour: retain the name, recover from invalid responses and stop on unavailable input

Read the depot name first, then the number of units requested. Keep prompts understandable. Ordinary information belongs on standard output; terminal input failures belong on standard error. Do not build a menu or a repeated dispatch system: one accepted request per run is sufficient.

| Input                     | Contract                                                                                                                                                                                             |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Depot name                | Read a complete line into a <code>std::string</code> , preserving spaces. A successfully read empty line is permitted. If no line can be read, report the failure and return a non-zero exit status. |
| Requested units           | An <code>int</code> from <b>0 to 60 inclusive</b> . Retry after an unreadable or out-of-range response.                                                                                              |
| Recovery                  | A retry must make progress past the failed response. Do not repeatedly attempt the same bad characters.                                                                                              |
| Ended/unavailable input   | Stop with an error rather than loop indefinitely.                                                                                                                                                    |
| Accepted numeric response | Use the <b>numeric-prefix policy</b> below and discard the remainder of that response line.                                                                                                          |

### Numeric-prefix policy

An accepted integer prefix is the request. For example, `25abc` means **25**, and the remaining `abc` is discarded. `abc25` has no readable integer prefix and requires another response. Likewise, `80abc` is outside the permitted range and requires a retry. A response such as `25 99` supplies one request, **25**, not two dispatches.

This is deliberately not strict whole-line number validation. Do not add a `new` parser that rejects an explicitly accepted response.

### Interactions your program must support

| Fresh responses, one per line                    | Required result                                       |
|--------------------------------------------------|-------------------------------------------------------|
| North Depot, then 25                             | Preserve the full name and accept 25.                 |
| North Depot, then abc, 80, 25                    | Two retries, then accept 25.                          |
| North Depot, then 25abc                          | Accept 25 and discard the rest of that line.          |
| A name, then input closes before a valid request | Report the ended input; no completed dispatch report. |

For input that closes immediately after an accepted valid integer, the request is still usable. You do not need any later console input. Write a two-sentence explanation of why resetting a stream's state and discarding input are different actions.

## B · Build the input handling in small steps

**Step 1 - Read the name first.** Use `std::getline` to read the complete depot name into a `std::string`. Check whether the read succeeded. An empty line is valid; failure to read a line is not.

**Step 2 - Start a request loop.** Keep asking until you have one integer in the range 0–60, or until input has ended. You do not need a menu and you do not need a second dispatch request.

**Step 3 - Attempt formatted integer input.** Use `std::cin >> requestedUnits` so that the published numeric-prefix policy is preserved. Do not replace it with strict whole-line parsing.

**Step 4 - Recover from unreadable input.** If integer extraction fails because of characters such as `abc`, first check whether input has ended. Otherwise reset the stream state, discard the remainder of that response line, and retry.

**Step 5 - Discard the rest of a readable response line.** After a readable integer prefix, discard the rest of that line before deciding whether to accept or retry. This makes `25abc` and `25 99` behave exactly as specified.

**Step 6 - Apply the range rule.** Accept 0–60 inclusive. If the readable integer is outside that range, report the problem and ask again. If a valid integer was already extracted and the line then ends at EOF, keep that valid request.

**Useful stream operations:** These are building blocks, not a complete solution. Include `<limits>` if you use `numeric_limits`.

```
std::getline(std::cin, depotName)
std::cin >> requestedUnits
std::cin.clear()
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n')
std::cin.eof()
```

| Response line                              | What your program should do                                              |
|--------------------------------------------|--------------------------------------------------------------------------|
| 25                                         | Accept 25 and finish the request loop.                                   |
| 25abc                                      | Accept 25; discard <code>abc</code> .                                    |
| abc25                                      | No readable integer prefix: recover, discard the line and retry.         |
| 80abc                                      | Read 80, discard <code>abc</code> , reject it as out of range and retry. |
| 25 99                                      | Accept 25; discard the remaining 99.                                     |
| 25 followed immediately by<br>end-of-input | Keep 25; the accepted request is still usable.                           |

**Checkpoint B:** With depot name North Depot, the response sequence `abc`, `80`, `25abc` causes two retries and then leaves `requestedUnits` equal to 25. Your exact prompt and error wording may differ.

## C · Audit the batch file before reporting success

**25 marks · Required behaviour: a complete validated stock total, not a successful partial report**

Read `batches.txt` relative to the program's working directory. The supplied project sets that directory to its project folder when launched through Visual Studio. The physical location of the file matters; its appearance in Solution Explorer is not sufficient evidence that it can be opened.

| File rule           | Required interpretation                                                                                                                                                                     |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Format              | A sequence of whitespace-separated integer entries; spaces, tabs and blank lines are permitted. Use the taught formatted-integer input model, not a <code>new line</code> -oriented parser. |
| Number of entries   | <b>1–12 inclusive</b> . A thirteenth entry invalidates the audit.                                                                                                                           |
| Value of each entry | <b>0–40 inclusive</b> . Each entry represents units in one batch.                                                                                                                           |
| Low batch           | An entry <b>strictly below 10</b> , including zero.                                                                                                                                         |
| Normal end          | A final valid number may have a trailing newline or no trailing newline. Both are accepted.                                                                                                 |
| Invalid data        | A failed numeric extraction, an unrepresentable integer, an out-of-range entry or an excess entry cancels the audit.                                                                        |
| No entries          | An empty or whitespace-only file is a separate error, not a zero-unit valid audit.                                                                                                          |

Calculate the **batch count**, **original stock total** and **low-batch count**. Preserve the original stock total for the final report. The stated limits keep this total at most 480; no larger-number algorithm is needed.

### Errors must remain distinguishable

Distinguish an open failure, a file with no entries, and invalid batch data. Also stop if the input stream reports a serious read failure. Error wording is flexible, but the cause must be recognisable. Return a non-zero status and do not print a completed stock/dispatch summary after any of these failures. A heading or prompt printed earlier is not a completed summary.

The file does **not** use the console's discard-the-rest-of-the-line policy. For example, `12abc 8` must **not** become a valid two-batch file by silently discarding `abc`. Do not treat a failed final number such as `+` or a very large integer as a clean end merely because the stream has reached the end of the file.

### Baseline data

The supplied `batches.txt` contains **12, 0, 8 and 20**, on separate lines. A valid audit reports **4 batches, 40 units and 2 low batches**. Other data sets are supplied in `data_cases`; they are inputs to test, not replacement implementations.

**Acceptance rule:** read and validate the whole file before applying a dispatch. An invalid later entry must not leave a successful-looking partial result.

## D · Dispatch through a selected quantity

25 marks · Required behaviour: correct business result and a traceable pointer-based update

Dispatch the smaller of the accepted request and original stock. A shortage dispatches the available stock; it is not a file error. A zero request dispatches nothing. Count non-empty dispatches, not units: this single-request program produces a count of zero or one.

### Required state and access constraints

Retain `totalStock` unchanged. Use a separate integer `availableStock`, initially equal to that total, and an integer `dispatchCount`, initially zero. Use **one non-owning `int*` named `selectedQuantity`** for the two updates below. A non-owning pointer accesses an existing object; it does not allocate or release it.

| Operation        | Requirement                                                                                                                                                  |
|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Stock update     | Select <code>availableStock</code> and subtract the dispatched amount <b>through the pointer</b> .                                                           |
| Dispatch counter | Retarget that same pointer to <code>dispatchCount</code> . Increment the pointed-to value once <b>only when</b> the dispatched amount is positive.           |
| End of selection | Set the pointer to <code>nullptr</code> . A guarded output path must report that nothing is selected without dereferencing it.                               |
| Safety           | Do not use <code>new</code> , <code>delete</code> , casts or arithmetic to jump between separate scalar variables. Do not execute an invalid pointer access. |

### Final report

Display the full depot name, batch count, low-batch count, original stock, requested units, dispatched units, remaining stock and dispatch count. Include a clear no-selection message after the final pointer reset. Labels and spacing may vary; numerical meaning must not.

| Data and request              | Dispatched | Remaining | Dispatch count |
|-------------------------------|------------|-----------|----------------|
| Baseline stock 40; request 25 | 25         | 15        | 1              |
| Baseline stock 40; request 0  | 0          | 40        | 0              |
| Stock 10 from 4 6; request 25 | 10         | 0         | 1              |
| Stock 0 from 0 0; request 25  | 0          | 0         | 0              |

### Trace your own execution

For the baseline run with request 25, record four states: before the stock update, after it, after retargeting and the possible increment, and after the null reset. At each state, identify the selected object, the relevant integer values, the pointer's stored value and its own address. Do not evaluate `*selectedQuantity` in the null state; record **not evaluated**.

Use Watch observations, a written table or an equivalent accessible trace. Explain why `selectedQuantity`, `*selectedQuantity` and `&selectedQuantity` are different expressions. Exact address numbers are not assessed. State why these objects do not require `delete`, and why a non-null value alone is not a universal safety guarantee.

## E · Demonstrate, explain and submit

15 marks · Required artefacts: a compact test record, explanations and reproducible source

Run the seven named checks below and **one additional test you design**, giving eight recorded tests. Predict each outcome before running it, then record what actually happened. A failed test is useful evidence when you explain the defect honestly; do not relabel an unrun test as passed.

| ID | Data / console input              | Acceptance condition                                                                           |
|----|-----------------------------------|------------------------------------------------------------------------------------------------|
| P1 | Baseline; North Depot, request 25 | Full name; 4 batches, low count 2, original 40, dispatched 25, remaining 15, dispatch count 1. |
| P2 | short-stock.txt; request 25       | 2 batches, low count 2, original 10, dispatched 10, remaining 0, count 1.                      |
| P3 | Baseline; request 0               | Original and remaining 40; dispatched 0, count 0.                                              |
| P4 | Baseline; requests abc, 80, 25abc | Recover twice, then accept 25 and produce the P1 quantities.                                   |
| P5 | batches.txt missing               | Open error, non-zero exit, no completed report.                                                |
| P6 | empty.txt                         | No-entries error, non-zero exit, no completed report.                                          |
| P7 | malformed-final.txt               | Invalid-data error, non-zero exit, no completed report.                                        |
| P8 | Your own case                     | Specify the input, the contract rule it tests and the expected result before running.          |

Additional checks may use whitespace-only input, an oversized entry, an out-of-range value, the maximum count, an excess entry, no final newline, a zero-stock file or ended console input. These are already published requirements, not undisclosed extensions. You are not required to cause a real hardware failure or perform an unsafe memory operation.

### Evidence to include

Use `evidence.md` or export the HTML notes into an equivalent document. Include the two diagnostic records and the build explanation from A; the input-recovery explanation from B; the pointer trace and safety explanation from D; and the eight test records above. For one test, explain why your chosen input detects a mistake that the baseline would miss. Brief tables are sufficient; a long report is not required.

**Suggested `evidence.md` structure:** Short sections are sufficient.

```
# Lab 01 evidence
## A - Build investigation (Fault 1, Fault 2, final output)
## A - Build explanation (<=80 words)
## B - Why clear() and discarding input are different
## D - Pointer trace and safety explanation
## E - P1-P8 test records and one targeted-test explanation
```

### Submission package

Restore the baseline `batches.txt`, confirm which projects build, and commit your final work. Push to the authorised private repository. Submit the **repository URL and final commit identifier**, your evidence file, and a source ZIP through the assessment's Blackboard submission point. The ZIP provides a reproducible snapshot, not an alternative to explaining your code.

Keep `.cpp`, `.h`, `.sln`, `.vcxproj`, `.gitignore`, the data file and evidence. Exclude generated binaries, `bin`, `obj`, `.vs`, per-user files and credentials. Record any unfinished requirement and its location. HTML ticks and notes do not submit code or award marks; export notes before closing a shared computer session.

### Final five-minute submission check

- ☐ `DispatchAudit` builds in the required configuration and the baseline run gives the expected quantities.
- ☐ `BuildInvestigation` builds and its newest executable prints `Packages: 3`.
- ☐ P1-P8 have a prediction and an actual recorded result; failed tests are identified rather than hidden.
- ☐ `batches.txt` has been restored to 12, 0, 8, 20 and `evidence.md` contains the required explanations/trace.
- ☐ Final source is committed and pushed; repository URL, final commit identifier, evidence and source ZIP are ready for Blackboard.

# How the work is assessed

## Mark allocation

| Task                        | Marks      | Evidence considered                                                                                  |
|-----------------------------|------------|------------------------------------------------------------------------------------------------------|
| A · Structure and diagnosis | 15         | Project/interface structure 5; two diagnosed repairs 6; build explanation 4                          |
| B · Console input           | 20         | Full-line name 4; range and recovery 8; ended input 4; numeric-prefix policy 4                       |
| C · Batch audit             | 25         | Valid aggregates 10; invalid/range/count handling 10; distinct missing/empty/read-failure handling 5 |
| D · Dispatch and pointers   | 25         | Dispatch quantities 8; pointer-based updates 9; trace, reset and safety explanation 8                |
| E · Evidence and submission | 15         | Eight test records 7; targeted reasoning/readability 4; reproducible submission and Git evidence 4   |
| <b>Total</b>                | <b>100</b> | Partial credit is awarded within each criterion.                                                     |

## What different levels of understanding look like

| Level               | Typical evidence                                                                                                                                                            |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Basic</b>        | A clear implementation of the normal case; genuine pointer use; some correct explanation. Recovery or boundary handling may be incomplete.                                  |
| <b>Intermediate</b> | Normal and common failure cases handled consistently; accurate build diagnosis; a trace that separates the pointer from its target.                                         |
| <b>Expert</b>       | The full published contract is respected, including less obvious endings and zero cases; no unsafe access; tests and explanations identify why plausible alternatives fail. |

These descriptors are not automatic percentage bands. A label, many commits or an attractive interface does not replace evidence. No extra library, optional tutorial extension or `new` language feature is needed for full marks.

## Official documentation

Use these as reference material under the assessment's published conditions. They are not a prescribed solution sequence.

[Microsoft — translation units and linkage](#) · [input-stream operations](#) · [input file streams](#) · [raw pointers](#).